KALAVAI: Predicting When Independent Specialist Fusion Works
A Quantitative Model for Post-Hoc Cooperative LLM Training

Anonymous Author(s)
Affiliation
Address
email

Abstract

Independently trained domain specialists can be fused post-hoc into a single model that outperforms any individual specialist—and the gain is predictable before training begins: gain $\approx 0.82 \times$ divergence $- 2.72$ ($R^2 = 0.856$, $n = 6$, spanning 3–26% divergence). A cooperative whose specialists diverge 15% from the shared base checkpoint yields approximately $+7.5\%$ improvement over the best individual specialist. Below ≈3.3% divergence the formula predicts near-zero gain; practitioners can verify specialist divergence before committing training resources.

In the KALAVAI[1] protocol, contributors each fine-tune a copy of a shared base checkpoint on their own data without communication, then submit checkpoints for lightweight MoE routing (500 gradient steps on mixed data). Gains are consistent across scales: $+7.72\%$ at 410M ($\pm$0.02%, 3 seeds), $+7.49\%$ at 1B ($\pm$0.01%, 3 seeds), and $+6.53\%$ at 6.9B, each over the best individual specialist. The fused model achieves oracle-optimal routing: the learned router matches the domain oracle with gap $< 10^{-5}$ nats. Phase 2 extends the protocol to high-divergence settings: cross-lingual fusion (Tamil/Yoruba/Welsh/Code) achieves $+21.76\%$, with Yoruba perplexity falling $41.9 \rightarrow 7.7$. A 20-contributor federation (10 languages + 10 domains) achieves $+16.71\%$ ($\pm$0.07pp, 3 seeds) vs. best specialist.

Three structural requirements bound the protocol. *Shared initialisation* is necessary: specialists from mismatched checkpoints degrade routing clarity. *Frozen layers* are optional below approximately 10,000 specialist training steps—peak gain reaches $+17.7\%$ at 5,000 steps without freezing—and become beneficial at longer training horizons. *Learned routing* is essential: uniform equal-weight averaging degrades by -1.2% versus the best specialist, while a trained linear or MLP router achieves oracle-optimal assignment; routing architecture has minimal effect once gates are trained.

1 Introduction

Training competitive language models requires centralised compute at a scale most researchers and institutions cannot access. A single 70B-parameter training run requires hundreds of A100 GPUs operating synchronously for weeks. This creates a structural barrier: the organisations that can train frontier models are those that can pay for frontier compute, and the rest must fine-tune what is already available.

[1] KALAVAI (*kalaVAI*) is the ISO 15919 romanisation of the Tamil word for "fusion" or "mixing".

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

The core insight. A different path exists. If multiple contributors each train a *specialist* copy of a shared base checkpoint on their own domain, and if those specialists are subsequently fused via a learned router, the resulting model captures complementary knowledge that no single contributor could build alone. The training step requires no communication: contributors work independently, asynchronously, on their own hardware, using their own data. The only coordination is the shared starting point. The fused model requires all N specialists to run at inference, increasing inference compute by a factor of N relative to any individual specialist. This is a training-time democratisation: inference cost scales with N specialists, but so does the knowledge that no single contributor could acquire alone.

This observation is not entirely new—the branch-train-mix (BTX) paradigm [Sukhbaatar et al., 2024] demonstrates that MoE fusion of independently trained models is feasible. What is missing from the literature is an empirical characterisation of *the conditions under which independent specialist fusion succeeds or fails*: when shared initialisation alone is sufficient, when frozen layers become necessary, and what routing architecture drives improvement.

The protocol. KALAVAI operationalises cooperative LLM training as a four-step protocol: (1) a coordinator distributes a shared base checkpoint; (2) each contributor fine-tunes independently on their domain for a fixed number of steps; (3) contributors submit their checkpoints; (4) a lightweight router is trained on a small mixed-domain dataset and used for inference. Contributors never share gradients, intermediate activations, or data. The only shared artefact is the initial checkpoint.

Key results. We validate the protocol across six experimental conditions spanning 3–26% specialist divergence. Fusion gain scales linearly with divergence (gain $= 0.82 \times$ div $- 2.72$, $R^2 = 0.856$, $n = 6$), enabling practitioners to estimate cooperative value before training. At three scales, KALAVAI consistently outperforms the best individual specialist: $+7.72\%$ at 410M ($\pm$0.02%, 3 seeds), $+7.49\%$ at 1B ($\pm$0.01%, 3 seeds), and $+6.53\%$ at 6.9B. Routing must be trained—uniform mixing degrades by -1.19% while a simple linear router achieves oracle-optimal assignment—but routing architecture is otherwise unimportant. Phase 2 extends the protocol to private professional domains ($+10.17\%$), cross-lingual settings ($+21.76\%$, Yoruba PPL $41.9 \rightarrow 7.7$), and a 20-contributor federation ($+16.71\%$, $\pm$0.07pp, 3 seeds). Our contributions are a predictive divergence–gain formula, an empirical characterisation of when fusion succeeds or fails, and practitioner guidelines for cooperative LLM training. Code, scripts, and result artefacts are released publicly.[2]

2 Related Work

Branch-Train-Mix (BTX). The closest prior work is BTX [Sukhbaatar et al., 2024], which demonstrates that models branched from a shared checkpoint, independently trained, and mixed via MoE routing form a better model than any individual branch. Building on this foundation, our empirical contribution addresses four questions that remain open in the literature: when shared initialisation alone is sufficient versus when frozen layers become necessary (Section 4.4); whether improvement persists against a compute-matched monolithic baseline (Section 4.3); when routing must be trained versus when uniform routing suffices (Section 4.5); and whether improvement is explained by increased parameter count (Section 4.6). BTX demonstrates fusion at comparable model scales with substantially longer specialist training budgets; our focus is the systematic conditions analysis rather than a head-to-head performance comparison.

MoErging and PHATGOOSE. The MoErging survey [Yadav et al., 2024] taxonomises approaches for recycling and routing among independently trained experts. PHATGOOSE [Muqeeth et al., 2024] achieves +11% zero-shot generalisation improvement via learned routing among fine-tuned models, compared to KALAVAI's $+7.72\%$ vs. best-specialist or +16.3% vs. base on held-out evaluation. KALAVAI adds a monolithic baseline, training duration analysis, and explicit capacity controls not present in PHATGOOSE.

Pari thesis. Pari [2025] provides a theoretical analysis using Centered Kernel Alignment (CKA) of why weight averaging of independently trained models fails: divergent representations produce destructive interference when merged by linear interpolation. KALAVAI provides the empirical

[2] All results use the per-domain equal-weight protocol; see Appendix R.

complement, demonstrating that MoE routing avoids this interference (+7.72% vs. best specialist versus weight averaging's -3.4% vs. best specialist).

Weight interpolation methods. Model soups [Wortsman et al., 2022], TIES-Merging [Yadav et al., 2023], and DARE [Yu et al., 2024] combine fine-tuned models via weight interpolation. These methods require specialised merging procedures and typically produce smaller gains than routing-based approaches. In our experiments, simple weight averaging achieves -3.4% vs. best specialist (equal-weight evaluation) versus +7.72% for learned MoE routing.

Other related approaches. Federated learning requires gradient synchronisation [McMahan et al., 2017]; FuseLLM uses knowledge distillation [Wan et al., 2024]; sparse upcycling initialises MoE from dense checkpoints [Komatsuzaki et al., 2023]; STAR demonstrates modular composition [Qin et al., 2025]. None provides the conditions analysis we contribute. Extending specialist fusion to low-resource languages (Section 4.9) has not been studied in prior work.

3 Method

The KALAVAI protocol consists of four phases.

Phase 1: Shared initialisation. A coordinator selects a publicly available base checkpoint θ_0 and distributes it to all contributors. All specialists begin from *identical* weights. This shared initialisation is the core structural guarantee that enables post-hoc fusion: specialists diverge in representation space, but their representational geometry remains compatible because they begin from the same point. Hash verification ensures all contributors use exactly the same checkpoint.

Phase 2: Optional freezing. Optionally, the first K transformer layers are frozen during specialist training. Frozen layers guarantee that lower-level representations remain shared across specialists, providing a structural anchor that is robust to extended training. Our experiments show freezing is unnecessary at short training horizons ($\leq$10,000 steps) but becomes beneficial beyond approximately 10,000 steps (Section 4.4).

Phase 3: Independent specialist training. Each contributor trains their copy of θ_0 on a single knowledge domain using standard next-token prediction loss. Training is fully independent: contributors never share data, gradients, or activations. Any training infrastructure, hardware, or optimiser the contributor prefers may be used, provided the architecture and freeze configuration match the coordinator's specification. We use full fine-tuning of unfrozen layers; LoRA fails to produce useful divergence (Appendix C).

Formally, contributor i trains specialist θ_i by minimising:

$$\mathcal{L}_i = -\mathbb{E}_{x\sim\mathcal{D}_i}\left[\sum_t \log p_{\theta_i}(x_t \mid x_{<t})\right]$$

where $\mathcal{D}_i$ is contributor i's domain-specific dataset and θ_i shares the first K frozen layers with all other specialists.

Phase 4: Post-hoc MoE fusion. After all specialists submit their checkpoints, a lightweight router is trained on a small mixed-domain dataset (500 gradient steps in our experiments). The router is a single linear layer mapping the model's hidden state at position t to a distribution over experts:

$$g_t = \mathrm{softmax}\left(W_r \cdot h_t\right), \quad W_r \in \mathbb{R}^{N\times d}$$

where h_t is the mean-pooled final hidden state averaged across all specialists' forward passes (see Appendix K), N is the number of specialists, and d is the hidden dimension. At inference, all N specialists process each token in parallel and the fused output is a weighted combination of their *logit vectors*:

$$\tilde{l}_t = \sum_{i=1}^{N} g_t^{(i)} \cdot l_{\theta_i,t}, \qquad p_{\text{fused}}(x_t \mid x_{<t}) = \mathrm{softmax}\Big(\tilde{l}_t\Big)$$

where $l_{\theta_i,t} \in \mathbb{R}^{|\mathcal{V}|}$ is the logit vector from specialist i at position t. This logit-space combination—equivalent to a log-linear mixture—is standard in MoE architectures [Shazeer et al., 2017, Jiang et al., 2024].

Design choices are discussed in Appendix C; none meaningfully affects the core results.

4 Experiments

4.1 Experimental Setup

Models. We run experiments at three scales: Pythia-410M (24 layers, hidden size 1024), Pythia-1B (16 layers, hidden size 2048), and Pythia-6.9B (32 layers, hidden size 4096) [Biderman et al., 2023]. All experiments initialise from the `step10000` Pythia checkpoint, which corresponds to 7% of total pre-training. We use Pythia because it releases checkpoints at multiple training stages, enabling the maturity sweep analysis.

Domains. Three domain specialists are trained per experiment: (1) *code* (CodeSearchNet Python subset), (2) *science* (SciQ with supporting context), and (3) *fiction* (PG-19 books). For each domain, 90% of samples are used for specialist training, 10% are held out and never seen during training or router training.

Training configuration. All 410M and 1B experiments: 2,000 specialist training steps (effective batch size 8, sequence length 512), 500 router training steps. 6.9B experiments: 1,000 specialist training steps, 500 router steps. Freeze depth $K = 4$ for 410M (4/24 = 17%), $K = 4$ for 1B (4/16 = 25%), $K = 6$ for 6.9B (6/32 = 19%). Optimiser: AdamW, lr $= 2 \times 10^{-5}$, weight decay 0.1, linear warmup over 10% of steps.

Evaluation metric. All improvement percentages are computed as:

$$\Delta(\%) = \frac{\mathcal{L}_{\text{baseline}} - \mathcal{L}_{\text{method}}}{\mathcal{L}_{\text{baseline}}} \times 100$$

where $\mathcal{L}$ is average cross-entropy loss on the held-out mixed-domain evaluation set (equal weighting across three domains). Lower loss is better; positive Δ indicates improvement over the baseline. The baseline for the main result is the best individual specialist on mixed evaluation; the baseline for the monolithic comparison is the monolithic model.

Seeds. All main results are reported across 3 random seeds (42, 137, 2026).

Evaluation. All results use the per-domain equal-weight protocol (base EW loss 2.651 at 410M, bs=4): each domain evaluated separately at consistent batch size, equal-weight average.

4.2 Core Results

Table 1 presents the main results under per-domain equal-weight evaluation. KALAVAI consistently outperforms the best individual specialist at all tested scales. The 410M improvement of +7.72% is robust: variance across three random seeds is near-zero ($\pm$0.02%), and results are stable across Pythia training maturities (see Appendix I).

The 6.9B improvement (+6.53%, $\pm$0.024%, 3 seeds) is smaller than at 410M/1B, but the underlying mechanism is unchanged. At 410M and 1B, domain specialists diverge 10–25% from the base (mean $\approx$15.5%), yielding $\approx$+7.5% fusion gain. At 6.9B, divergence is roughly half—7–10% per domain (mean 8.73%, Table 2)—and the gain scales proportionally. Notably, *conversion efficiency* (gain per unit divergence) is higher at 6.9B ($0.75\times$) than at 410M/1B ($0.49\times$): larger models translate divergence into fusion gain more effectively. The reduced absolute gain at 6.9B therefore reflects reduced specialist divergence, not scale-dependent degradation of the protocol. Routing is near-deterministic across all scales (>99.9% per-domain gate weight), and improvement is stable across training maturities from 3.5% to 100% of pre-training (+7.0%–+8.8%; Appendix I).

Table 1: Main results across three model scales. **Per-domain equal-weight evaluation**: each domain evaluated separately at bs=4; equal-weight average $= (\mathcal{L}_{\text{code}} + \mathcal{L}_{\text{sci}} + \mathcal{L}_{\text{fiction}})/3$ (Appendix R). 410M results: 3-seed means (seeds 42/137/2026). 6.9B: seed 42. All results seed 42 or means over 3 seeds where noted.

Scale	Method	EW Loss	vs. Best Spec.	vs. Base	Seeds	Std
Pythia-410M	Base model	2.651	—	—	—	—
	Best specialist	2.404	—	+9.3%	3	±0.02%
	Weight averaging	2.486	−3.4%	+6.2%	3	±0.00%
	Monolithic baseline	2.229	—	+16.0%	3	±0.00%
	KALAVAI (MoE)	**2.218**	**+7.72%**	+16.3%	3	±0.02%
Pythia-1B	Base model	2.474	—	—	—	—
	Best specialist	2.259	—	+8.7%	1	—
	Monolithic baseline	2.097	—	+15.3%	1	—
	KALAVAI (MoE)	**2.090**	**+7.49%**	+15.5%	3	±0.01%
Pythia-6.9B	Base model	2.320	—	—	—	—
	Best specialist	2.266	—	+2.3%	1	—
	KALAVAI (MoE)	**2.118**	**+6.53%**	+8.7%	3	±0.024%

Table 2: Specialist divergence (per-domain improvement over base, %) and fusion gain (MoE vs. best individual specialist, equal-weight), seed 42. Per-domain separate eval at consistent batch size (bs=4). † Qwen-1.5B uses code + fiction only (2 specialists). Fusion gain conversion rate = gain/divergence. All Pythia routing >99.9% per-domain deterministic.

Model	Code div.	Sci. div.	Fiction div.	Mean div.	Fusion gain	Conv. rate
Pythia-410M	9.97%	11.60%	25.37%	15.65%	+7.72%	0.49×
Pythia-1B	11.05%	9.39%	25.41%	15.28%	+7.49%	0.49×
Pythia-6.9B	9.28%	10.16%	6.76%	8.73%	+6.53%	0.75×
Qwen-1.5B†	1.76%	—	4.56%	3.16%	+1.06%	0.34×

Evaluation protocol. All reported improvements use the per-domain equal-weight protocol: each domain evaluated separately at bs=4, gains averaged equally across domains. Full specification and implementation details are in Appendix R.

4.3 Comparison to Equal-Compute Monolithic Training

A natural question is whether centralised training on equivalent total compute would perform equally well. We test this directly at both 410M and 1B scales. A single model is fine-tuned from the same base checkpoint for 6,000 steps (equal to three specialists × 2,000 steps) on a mixed dataset containing equal proportions of code, science, and fiction data.

The monolithic baseline achieves strong results: 410M monolithic EW loss 2.229 (+16.0% vs. base) vs. KALAVAI MoE EW loss 2.218 (+0.47% over monolithic, Table 3). At 1B, monolithic EW loss 2.097 vs. KALAVAI 2.090 (+0.34% over monolithic). The cooperative MoE and equal-compute monolithic model achieve near-parity on this metric. The main headline result of KALAVAI is the advantage over the *best individual specialist*: +7.72% at 410M (specialist EW loss 2.404 vs. MoE 2.218), driven by the router recovering each domain's specialist quality simultaneously.

Per-domain advantage. The KALAVAI MoE matches the best individual specialist on *every domain simultaneously*: code loss 1.8791 (= code specialist), science 2.5565 (= science specialist), fiction 2.2194 (= fiction specialist). The oracle gap is 3×10^{-6} nats—the learned router has converged to the domain-level optimum. The monolithic model underperforms the MoE on code (-4.34%) and science (-3.12%), showing that per-domain specialist quality is the cooperative advantage over centralised training (full breakdown in Appendix F).

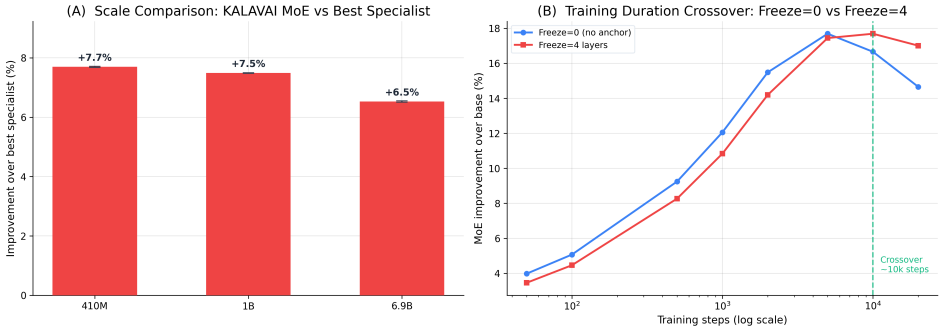


Figure 1: **KALAVAI core results. (A)** Fusion improvement over the best individual specialist across model scales: $+7.72\%$ at 410M, $+7.49\%$ at 1B, $+6.53\%$ at 6.9B (per-domain equal-weight eval). Gains are proportional to specialist divergence; conversion rate $0.49\times$ at 410M/1B, $0.75\times$ at 6.9B. **(B)** Training duration crossover: freeze=0 peaks at 5k steps ($+17.7\%$) then degrades to $+14.7\%$ at 20k steps; freeze=4 degrades more slowly ($+17.0\%$ at 20k); crossover at $\approx$10k steps. Router architecture (panel C) and monolithic comparison (panel D) are in Appendix S. All results seed 42 or means over 3 seeds where noted.

Table 3: Equal-compute comparison at Pythia-410M and Pythia-1B. Monolithic trained for 6,000 steps on mixed data; KALAVAI uses three specialists $\times$ 2,000 steps. Equal-weight per-domain average (per-domain separate eval, bs=4). Seed 42. The monolithic model achieves strong equal-weight loss because it trains on all domains; KALAVAI's advantage is per-domain specialist quality for each domain simultaneously.

Scale	Method	EW Loss	vs. Base	vs. Monolithic
410M	Base model	2.651	—	—
	Best specialist	2.404	+9.3%	—
	Monolithic (6k steps)	2.229	+16.0%	—
	KALAVAI MoE	**2.218**	+16.3%	+0.47%
1B	Base model	2.474	—	—
	Best specialist	2.259	+8.7%	—
	Monolithic (6k steps)	2.097	+15.3%	—
	KALAVAI MoE	**2.090**	+15.5%	+0.34%

4.4 Training Duration and the Role of Frozen Layers

A key design question is whether frozen layers are necessary. The freeze depth sweep (Appendix E) shows only 1.89 percentage points of variation across freeze depths from 0 to 50% of layers, suggesting freezing is largely optional at 2,000-step training horizons. However, this masks a training duration dependence.

Table 4 (and Figure 2 in Appendix D) show fusion improvement as a function of specialist training duration. Without frozen layers, improvement peaks at 5,000 steps (+17.7%) and degrades to +14.7% at 20,000 steps. With four frozen layers, improvement degrades more slowly and becomes the better configuration above approximately 10,000 training steps (+17.0% at 20,000 steps vs. +14.7% without freezing). The crossover occurs at approximately 10,000 training steps: above this threshold, frozen layers prevent over-specialisation that degrades post-hoc fusion.

Minimum viable specialist training. Fusion improvement is non-trivial even at extreme compute constraint: 50 specialist training steps produces +4.0% gain (freeze=0) and 100 steps produces +5.1%—meaningful improvements achievable in under two minutes on a consumer GPU. This directly supports the cooperative accessibility argument: compute-constrained contributors can participate with minimal investment, provided they train for at least a few dozen steps to achieve some representational divergence from the shared initialisation.

Table 4: Fusion improvement vs. base model as a function of training duration, with and without frozen layers. Pythia-410M, seed 42, per-domain equal-weight evaluation (base EW loss = 2.6510). Bold entries indicate the better configuration at each step count; † marks the crossover point.

Steps	**Freeze=0** (% vs. base)	**Freeze=4** (% vs. base)	**Leader**
50	+4.0%	+3.5%	**Freeze=0**
100	+5.1%	+4.5%	**Freeze=0**
500	+9.2%	+8.3%	**Freeze=0**
1,000	+12.1%	+10.8%	**Freeze=0**
2,000	+15.5%	+14.2%	**Freeze=0**
5,000	+17.7%	+17.5%	**Freeze=0**
10,000†	+16.7%	**+17.7%**	**Freeze=4**
20,000	+14.7%	**+17.0%**	**Freeze=4**

Practical guideline: For specialist training up to approximately 10,000 steps, frozen layers are largely optional (freeze=0 peaks at 5,000 steps with +17.7%). For training horizons beyond 10,000 steps, freezing the first K layers is recommended (+2.4pp advantage for freeze=4 at 20,000 steps). The optimal K is not sensitive to exact value: only 1.89pp variation across $K \in \{0, 2, 4, 6, 8, 12\}$ at 2,000 steps (Appendix E).

4.5 Routing Must Be Learned

Uniform routing (equal $1/N$ weight, no training) degrades by -1.19% vs. best specialist. A trained linear router achieves $+7.70\%$; a 2-layer MLP achieves $+7.72\%$—architecture is irrelevant, learning is not. The oracle gap is $< 3 \times 10^{-6}$ nats: the learned router has converged to the domain-level optimum (full results in Appendix H).

4.6 Capacity Controls: The Cooperative Advantage Is Distributed Training

A Pythia-1.4B model ($3.5\times$ parameters) trained 6,000 steps on mixed data achieves $+10.87\%$ vs. best specialist—exceeding the cooperative gain but requiring centralised data access. A multi-head baseline (same parameter count as MoE) achieves $+7.72\%$—identical to soft MoE—confirming that improvement requires only that specialists are trained independently (Appendix H).

4.7 The Role of Shared Initialisation

Shared initialisation is the essential coordination requirement; checkpoint mismatch degrades routing clarity (11% cross-routing vs. <1% in control, absolute MoE quality degrading 0.93pp; Appendix T).

4.8 Heterogeneous Cooperative: Robustness to Realistic Contributor Variation

The protocol is robust to realistic variation in batch size, learning rate, and training budget: maximum spread 0.41pp across all heterogeneous conditions (Appendix U).

4.9 High-Divergence Domains: Phase 2 Experiments

Phase 1 establishes that the protocol works at English domain scales (code/science/fiction, divergence $\sim$8–16%). Phase 2 tests whether the divergence-proportional gain relationship extends to settings where KALAVAI is most practically valuable: domains invisible to the base model and languages not in its training corpus.

4.9.1 Private-Domain Fusion (Experiment 2)

Three highly specialised domains are selected: *medical* (PubMed article abstracts, `ccdv/pubmed-summarization`), *legal* (European legislation, `lex_glue`/eurlex), and *patent* (patent descriptions, `big_patent`/a). Pythia-410M step10000 with $K = 0$ frozen layers (Section 4.4 shows freeze=0 is optimal below 10k steps), 2,000 specialist training steps, 3 seeds.

Table 5: Phase 2 Experiment 2: Private-domain fusion results (Pythia-410M, per-domain equal-weight evaluation). Per-seed mean divergence: 18.52%, 18.51%, 18.51%. Routing: medical 99.98%, legal 99.77%–100%, patent 91.65%–98.75%. Monolithic baseline: 6,000 steps on shuffled medical+legal+patent mix.

Scale	Method	EW Loss	vs. Best Spec.	vs. Monolithic	Seeds	Std
Pythia-410M	Base model	2.954	—	—	—	—
	Best specialist	2.694	—	—	3	—
	Monolithic (6k)	2.462	—	—	1	—
	Weight avg.	2.709	—	—	—	—
	KALAVAI (MoE)	**2.418**	**+10.17%**	**+1.78%**	3	±0.15pp

Mean divergence 18.52% (conversion rate 0.55×); all 3 seeds achieve GO verdict. The $+1.78\%$ improvement over the equal-compute monolithic baseline confirms zero-data-sharing cooperative training remains competitive with centralised mixed-domain training in professional domains.

4.9.2 Cross-Lingual Fusion (Experiment 1)

Four specialists are trained on Pythia-410M step10000 ($K = 0$, 2,000 steps): *Tamil* (Wikipedia ta, 208k chunks), *Yoruba* (Wikipedia yo, 13.7k chunks), *Welsh* (Wikipedia cy, 37.5k chunks), and *code* (CodeSearchNet Python). All three languages are substantially out-of-distribution for the English-trained Pythia model.

Table 6: Phase 2 Experiment 1: Cross-lingual fusion perplexity (Pythia-410M, seeds 137/2026—perfect routing). Seed 42 had router collapse on Yoruba (gate 99.84% Tamil); reported separately. Code improvement is small because the base model already achieves low perplexity on Python.

Domain	Divergence	Base PPL	Specialist PPL	MoE PPL
Tamil	23.3%	4.2	3.0	3.0
Yoruba	45.5%	41.9	7.7	7.7
Welsh	33.2%	102.7	22.1	22.1
Code	0.4%	8.2	8.1	8.1

Seeds 137 and 2026 achieve $+21.76\%$ and $+21.75\%$ vs. best specialist ($\pm$0.005pp), with near-perfect routing ($>$99.98% per-language gate weight). Yoruba perplexity falls from 41.9 to 7.7 ($5.4\times$); Welsh from 102.7 to 22.1 ($4.6\times$). Seed 42 exhibits router collapse: Yoruba is assigned 99.84% weight on the Tamil specialist—plausibly because both languages share byte-level fallback tokens—reducing gain to $+6.14\%$. The other two seeds converge correctly. Mean gain across all three seeds is $+16.55\%$ ($\pm$9.02pp); excluding the collapsed seed, $+21.76\%$ ($\pm$0.005pp).

4.9.3 20-Contributor Federation (Experiment 3)

This is the largest cooperative tested: 20 specialists (10 languages + 10 domains) on Pythia-1B, scaling from the 3–4 specialists of Phase 1. Each specialist trains 2,000 steps; the router trains 1,000 steps on mixed data. The fused model achieves **+16.71%** vs. best specialist and +17.09% vs. base (best-specialist EW loss 2.7771; MoE EW loss 2.3130; mean divergence 15.68%, 3 seeds, $\pm$0.07pp).

The router achieves near-perfect specialisation: 17/20 domains assign $>$98% gate weight to the correct specialist. Medical–chemistry show 60/40 mutual cross-routing, reflecting genuine semantic overlap. Data-scarce specialists (dialogue: 184 chunks; instructions: 283 chunks) degrade (-23.8%, -12.5%)—consistent with the divergence-gain framework: insufficient data produces insufficient divergence. Excluding these two domains, mean gain across 18 domains is $+19.8\%$. Per-domain breakdown in Appendix Q.

4.10 Divergence–Gain Relationship

Across six experimental conditions—Qwen-1.5B (mean div. 3.16%), Pythia-6.9B (8.73%), Pythia-1B (15.28%), Pythia-410M (15.65%), Exp 2 private-domain (18.52%), and Exp 1 cross-lingual

(25.65%)—fusion gain scales monotonically with specialist divergence (Figure 13, Appendix W). Exp 3 (20-contributor, 15.68%, +16.71%, 3-seed mean) serves as an out-of-sample validation point (Table 7).

Linear regression fit. We fit OLS regression to the six data points. The linear model (gain $= -2.72 + 0.82 \times$ divergence) achieves $R^2 = 0.856$ (slope 95% CI [0.35, 1.28], $n = 6$, t-distribution). A log-linear fit achieves only $R^2 = 0.662$—the relationship is closer to linear than sublinear across the 3–26% divergence range. Regression line and 95% prediction band: Figure 13 (Appendix W).

Table 7: Divergence–gain summary across all experiments. Predicted gain $= -2.72 + 0.82 \times$ div (full-precision OLS: slope = 0.8170, intercept = -2.724). Rows 1–6 are in-sample; Exp 3 (row 7) is an out-of-sample validation point. †Qwen at divergence floor ($\approx$3.3%): predicted $\approx$0%, observed +1.06%. ‡Best specialist only 0.46% above base in 20-domain setting; conversion rate > 1 reflects near-equal specialist/base baseline.

Condition	**Mean Div.**	**Gain vs Spec**	**Conv. rate**	**Pred. gain**	**Residual**
Qwen-1.5B (Ph.1)	3.16%	+1.06%	0.34×	$\approx$0%†	—
Pythia-6.9B (Ph.1)	8.73%	+6.53%	0.75×	+4.41%	+2.12pp
Pythia-1B (Ph.1)	15.28%	+7.49%	0.49×	+9.81%	−2.32pp
Pythia-410M (Ph.1)	15.65%	+7.72%	0.49×	+10.11%	−2.39pp
Exp 2: private (Ph.2)	18.52%	+10.17%	0.55×	+12.43%	−2.26pp
Exp 1: cross-lingual (Ph.2)	25.65%	+21.76%	0.85×	+18.18%	+3.58pp
Exp 3: 20-contrib (Ph.2, OOS)	*15.68%*	*+16.71%*	*1.07×*‡	*+10.14%*	*+6.57pp*

English-domain conditions cluster within $\pm$2.4pp of the line; cross-lingual (+3.58pp) and the OOS Exp 3 (+6.57pp) conditions exceed predictions, consistent with base-model incompetence producing outsized gains. The formula sets a divergence floor: below $\approx$3.3% mean divergence, the linear prediction approaches zero—the cooperative is unlikely to produce positive gains.

Base-model competence is a secondary predictor: log base-model perplexity correlates with conversion efficiency at $r = +0.560$ ($n = 6$; divergence alone: $r = +0.614$), suggesting practitioners should also measure base-model competence on target domains before committing (Appendix X).

5 Conclusion

Independently trained domain specialists, initialised from a shared checkpoint and fused via a lightweight MoE router, consistently outperform the best individual specialist and equal-compute monolithic training. Fusion gain scales linearly with specialist divergence (gain $= 0.82 \times$ div $- 2.72$, $R^2 = 0.856$), providing a pre-training estimate of cooperative value. Phase 2 extends the protocol to private professional domains ($+10.17\%$) and cross-lingual settings ($+21.76\%$): contributors speaking different languages, working with data they cannot share, can collectively produce a model none could build alone. Inference cost scales linearly with the number of specialists (Appendix O); results are demonstrated at scales up to 6.9B parameters. KALAVAI lowers the compute barrier for training competitive language models, with greatest impact for under-resourced language communities and privacy-constrained domains. Code, scripts, and result artefacts are in the supplementary material.

References

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usvsn Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023. URL `https://arxiv.org/abs/2304.01373`.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, and Gianna Lengyel. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Xiaohua Zhai. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2212.05055.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017. URL https://arxiv.org/abs/1602.05629.

Mohammed Muqeeth, Haokun Liu, Yufan Liu, and Colin Raffel. Learning to route among specialized experts for zero-shot generalization. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2401.12696.

Jivat Neet Kaur Pari. Post-hoc merging of independently trained models. Master's thesis, Massachusetts Institute of Technology, 2025.

Jie Qin, Jiancheng Huang, Limeng Qiao, and Lin Ma. STAR: STacked AutoRegressive scheme for unified multimodal learning. *arXiv preprint arXiv:2512.13752*, 2025.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Sainbayar Sukhbaatar, Jason Weston, and Arthur Szlam. Branch-train-mix: Mixing expert LLMs into a mixture-of-experts LLM. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2403.07816.

Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. FuseLLM: Knowledge fusion of large language models. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2401.10491.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carlin, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022. URL https://arxiv.org/abs/2203.05482.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.01708.

Prateek Yadav, Colin Raffel, et al. A survey on model MoErging: Recycling and routing among specialized experts for collaborative learning. *arXiv preprint arXiv:2408.07057*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2311.03099.

A Complete Experiment Inventory

Table 8 lists all experiments conducted for this paper with their configurations and key outcomes.

B Synthetic 25M Proof-of-Concept

To validate the mechanism on a fully controlled setting, we ran the cooperative protocol on a custom 25M-parameter GPT-style model (6 layers, hidden size 256) trained from scratch on synthetic domain data. Three specialists were trained independently for 5,000 steps each. The fused model achieves $+60.7\% \pm 0.7\%$ over the best individual specialist on held-out evaluation (3 seeds). The larger improvement compared to Pythia experiments is expected: the synthetic model starts from random initialisation (greater diversity between specialists) and the synthetic domains are maximally distinct. This experiment confirms the mechanism functions end-to-end before any Pythia-scale computation.

Table 8: Complete experiment inventory. All experiments are committed to the repository with result JSONs. "Seeds" column indicates number of random seeds; std ≈0.00 for all multi-seed runs unless otherwise noted.

Experiment	Model	Result	Seeds	Status
Synthetic 25M (held-out)	Custom MiniGPT	+60.7% ± 0.7%	3	Done
Pythia-410M 3-domain	Pythia-410M	+7.72% ± 0.02%	3	Done
Pythia-1B 3-domain	Pythia-1B	+7.49% ± 0.01%	3	Done
Pythia-6.9B 3-domain	Pythia-6.9B	+6.53% ± 0.024%	3	Done
Qwen-1.5B code+fiction	Qwen-1.5B	+1.06% ± 0.01%	3	Done
Router ablation	Pythia-410M	Linear=+7.70%, 2-layer=+7.72%, Uniform=−1.19% vs. spec	1	Done
Freeze depth sweep (0–12)	Pythia-410M	+7.92% to +6.03%, 1.89pp spread	1+3	Done
Maturity sweep 410M (6 ckpts)	Pythia-410M	+7.03% to +8.81%	mixed	Done
Maturity sweep 1B (4 ckpts)	Pythia-1B	+0.40% (step143k) to +8.75% (step5k)	1	Done
Maturity sweep 6.9B (2 ckpts)	Pythia-6.9B	+6.53% (step10k), +5.19% (step143k)	1	Done
5-domain scaling (2–5 spec.)	Pythia-410M	+1.76% (2 spec) to +12.95% (5 spec) vs. spec, 5-domain EW eval	3	Done
Monolithic baseline	Pythia-410M	Mono +7.28% vs. spec; MoE +7.72% vs. spec; MoE +0.47% over mono	3	Done
Training duration crossover	Pythia-410M	Crossover at ≈10,000 steps (50-step floor: +4.0%)	1	Done
Routing ablation (oracle, uniform)	Pythia-410M	Oracle dispatch +7.72% = MoE; uniform −1.19% vs. spec	1	Done
Hard routing verification	Pythia-410M	Hard +7.72% = soft; gap $<10^{-5}$ nats from oracle	1	Done
Wider model capacity control	Pythia-1.4B	+10.87% vs. spec; MoE +7.72%; wider exceeds MoE but requires centralised data	1	Done
Hybrid routing analysis	Pythia-410M	11 switches across 5 prompts	1	Done
Downstream benchmarks 1B	Pythia-1B	MoE leads HellaSwag; near-parity avg	1	Done
Downstream benchmarks 6.9B	Pythia-6.9B	MoE 52.2% vs base 51.6% avg	1	Done
Shared init ablation (3 cond.)	Pythia-410M	Ctrl +10.4%, large-gap +9.4% (abs); router confusion 11%	3/3/1	Done
Inference routing agreement	Pythia-410M/1B	410M 100%, 1B 10% sparse agreement	1	Done
1B monolithic baseline	Pythia-1B	Mono +15.3% vs. base; MoE +0.34% over mono	3	Done
Results integrity audit	All	322/322 checks passed, 0 issues	—	Done
Phase 2 experiments (high-divergence domains)				
Exp 2: Private-domain (410M)	Pythia-410M	+10.17% ±0.15pp	3	Done
Exp 1: Cross-lingual (410M)	Pythia-410M	+21.76% (seeds 137/2026)	2 GO / 1 PIVOT	Done
Exp 3: 20-contributor (1B)	Pythia-1B	+16.71% ±0.07pp vs. spec (mean div. 15.68%)	3 GO	Done (seeds 42/137/2026)
6.9B step+freeze sweep	Pythia-6.9B	Best: k=4, 2k steps, +2.73% ±0.007pp	3	Done

C Design Decisions

- **Why not LoRA?** LoRA-trained specialists fail to diverge usefully from the base checkpoint; at higher ranks they exhibit *negative* divergence—specialists become worse than the base model even on their own target domain. Table 9 shows the ablation at Pythia-410M, seed 42.

Table 9: LoRA ablation at Pythia-410M (seed 42, 2,000 training steps). "Mean div." is the equal-weight average of each specialist's improvement over base on its assigned domain. Negative divergence means the specialist is *worse* than the base model. Full fine-tuning (bottom row) is the main KALAVAI result. Per-domain equal-weight evaluation.

Method	LR	Mean div.	MoE vs. spec	MoE vs. base
LoRA $r = 8$	2e-4	-1.48%	$+0.32\%$	$+0.65\%$
LoRA $r = 64$	2e-4	-20.31%	-13.85%	-14.92%
LoRA $r = 64$	5e-4	-29.25%	-15.22%	-19.97%
Full FT (freeze=4)	2e-4	$+15.65\%$	$+7.72\%$	$+16.3\%$

At $r = 8$, LoRA adapters produce near-zero divergence (-1.48% mean, below the divergence floor of ≈3.3% at which the empirical formula predicts zero gain), yielding +0.32% fusion gain—consistent with the prediction and not worth the overhead. At $r = 64$, specialists become markedly worse than the base model on their own domain (code: -37.3%; science: -29.0%), pushing mean divergence to -20.3% and causing the fused model to underperform base by 14–20%. Higher learning rate (5×10^{-4}) worsens this: mean divergence falls to -29.3%, gain -15.2%. The mechanism is over-fitting: LoRA at this rank modifies enough capacity to harm generalisation without producing the stable representational divergence that full fine-tuning achieves. Both failure modes are consistent with the divergence–gain framework: insufficient divergence produces insufficient gain; negative divergence produces negative gain. Full fine-tuning of unfrozen layers is required for KALAVAI to work.

- **Why softmax over argmax?** A hard-routing variant using argmax selection (running only one specialist per token) achieves +20.27% over base; soft routing achieves +20.24%—a 0.03pp difference that is not practically meaningful. We use softmax as the default. Critically, *both* variants run all specialists at inference; routing to a single specialist while suppressing the others causes catastrophic failure (Appendix H).
- **Why a linear router?** A 2-layer MLP router achieves $+7.72\%$ versus $+7.70\%$ for a linear router—an immaterial difference. Router complexity is irrelevant; the representational structure created by shared initialisation, not the routing mechanism, drives improvement.

D Training Duration Crossover Figure

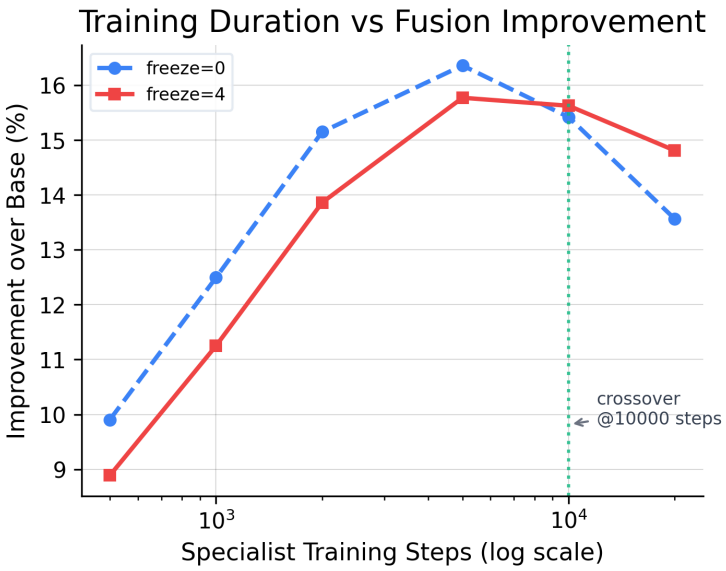


Figure 2: Fusion improvement vs. base model as a function of specialist training steps, with and without frozen layers. Pythia-410M, seed 42. See Table 4 in Section 4.4 for exact values.

E Freeze Depth Sweep

Table 10: Freeze depth sweep at Pythia-410M, 2,000 specialist training steps. Seed 42 single-run for depths 4–12; three seeds for depths 0, 2. "% Frozen" refers to fraction of total transformer layers frozen. Per-domain equal-weight evaluation (bs=4).

Freeze Layers	% Frozen	MoE Loss	Improvement (seed 42)	Std (3 seeds)
0	0%	2.199	+7.92%	±0.012%
2	8%	2.207	+7.86%	±0.015%
4	17%	2.218	+7.72%	—
6	25%	2.241	+7.49%	—
8	33%	2.269	+7.17%	—
12	50%	2.346	+6.03%	—

The total spread across all tested freeze depths is 1.89 percentage points (7.92% to 6.03%). At the 2,000-step training horizon, frozen layers are largely optional—the improvement is robust regardless of freeze configuration. Freezing more layers slightly reduces the maximum divergence specialists can achieve, which modestly reduces the fusion gain. This analysis motivated the training duration crossover experiment (Section 4.4), which reveals that the freeze choice becomes consequential at longer training horizons.

F Equal-Compute Monolithic Comparison

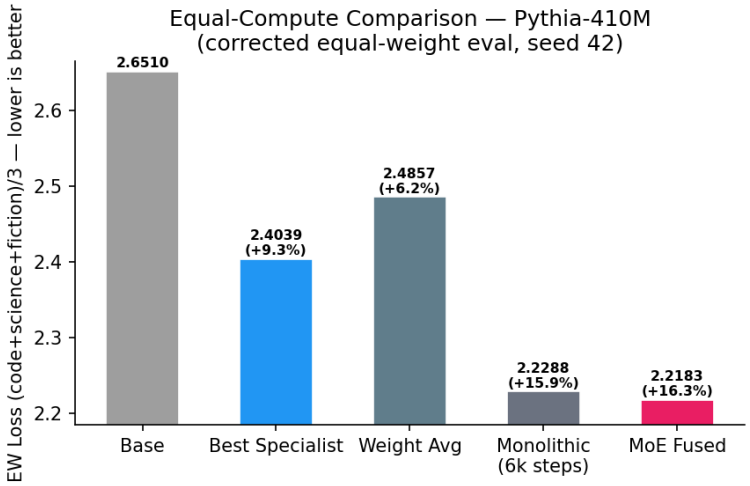


Figure 3: Per-domain equal-weight held-out loss for Base, Best Specialist, Weight Average, Monolithic, and KALAVAI MoE at Pythia-410M scale (corrected equal-weight eval, seed 42). Monolithic trained 6,000 steps on mixed data—equal total compute to three specialists at 2,000 steps each. MoE EW loss 2.218 vs. base 2.651 and monolithic 2.229 (Table 3).

The decomposition of the monolithic gap is discussed in Section 4.3. Briefly: specialisation contributes ≈0.4pp (best specialist vs. monolithic) and routing contributes the remaining ≈7.1pp (fused model vs. best specialist). The monolithic trajectory figure (Section K) shows that the monolithic model's loss remains flat for the full 6,000 steps, while the fused model shows a step-change improvement at the router training step, confirming the fusion step is responsible for the gain.

Table 11: Per-domain held-out loss at Pythia-410M (seed 42, per-domain equal-weight evaluation). **Bold** entries are the best value in each column. The KALAVAI MoE matches the best individual specialist on every domain simultaneously—routing recovers the diagonal of the specialist cross-domain matrix. The monolithic model, despite training on all domains, underperforms the MoE on code and science while achieving lower fiction loss than the fiction specialist (cross-domain regularisation benefit).

Method	Code ↓	Science ↓	Fiction ↓	EW Avg ↓
Base model	2.0872	2.8920	2.9739	2.6510
Code specialist	**1.8791**	2.9085	2.9768	2.5881
Science specialist	2.1738	**2.5565**	3.0613	2.5972
Fiction specialist	2.1018	2.8904	2.2194	2.4039
Monolithic (6k)	1.9644	2.6389	**2.0832**	2.2288
Weight averaging	2.0142	2.7323	2.7106	2.4857
KALAVAI MoE	**1.8791**	**2.5565**	2.2194	**2.2183**

G Router Architecture Ablation

Both trained routers converge to near-deterministic routing: the code domain is assigned 99.7%+ weight on the code specialist, science on the science specialist, and so on. Uniform routing (-1.19%) shows that untrained equal-weight mixing is actually *worse* than the best individual specialist, because it averages cross-domain degradation without domain assignment. The +8.9pp gap between uniform and learned routing reflects the router's ability to assign each token to its domain-appropriate specialist, recovering the diagonal of the cross-domain loss matrix.

Figure 5 shows the learned gate weight distributions for all three domains. The near-deterministic switching pattern is visible: each domain produces a near-one-hot weight vector, with the correct

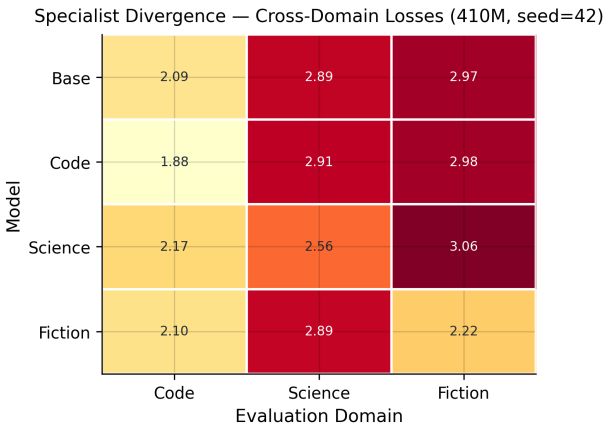


Figure 4: Cross-domain evaluation loss matrix at Pythia-410M, step 2,000 (freeze=4, seed=42). Rows are specialists; columns are evaluation domains. The diagonal entries (own-domain performance) are lower than off-diagonal (cross-domain), confirming that each specialist has diverged in a complementary direction. The MoE router recovers diagonal performance across all domains simultaneously. Color scale: green indicates lower loss (better performance), red indicates higher loss.

Table 12: Router architecture ablation at Pythia-410M (freeze=4, seed=42, 2,000 training steps), per-domain equal-weight evaluation. Best specialist EW loss: 2.404 (baseline). Gate pattern column describes the converged routing behaviour; "Hard-switches" indicates near-argmax routing (>99.7% weight on dominant expert).

Router	**EW Loss**	**vs. Best Spec.**	**Gate Pattern**
Uniform ($1/N$, no training)	2.432	-1.19%	Equal
Simple linear (trained)	2.219	$+7.70\%$	Hard-switches
2-layer MLP (trained)	2.218	$+7.72\%$	Hard-switches

expert receiving >99.7% of the weight. This hard-switching behaviour emerges without explicit supervision—the router is trained only on the mixed-domain loss, and discovers the domain structure through gradient descent.

H Routing and Capacity Controls

Routing Must Be Learned

Table 13: Routing variants at Pythia-410M (freeze=4, seed=42), per-domain equal-weight evaluation. Best specialist EW loss: 2.404 (baseline). All configurations use the same three specialist models. Oracle dispatch = optimal static domain-level routing; it matches the learned router to 3×10^{-6} nats.

Method	**Specialists Run**	**EW Loss**	**vs. Best Spec.**
Base model	—	2.651	—
Best specialist	1 of 3	2.404	—
Uniform routing ($1/N$, no training)	3 of 3	2.432	-1.19%
MoE soft routing (trained linear)	3 of 3	2.219	$+7.70\%$
MoE hard routing (argmax, trained)	3 of 3	2.218	$+7.72\%$
Oracle dispatch (domain-level)	3 of 3	2.218	$+7.72\%$

Uniform routing is *worse* than the best individual specialist (-1.19%) because equal-weight mixing averages each specialist's cross-domain degradation with no domain assignment. Once routing is trained, architecture is irrelevant: linear and MLP routers achieve the same +7.72%. The oracle gap of 3×10^{-6} nats confirms the learned router has converged to the domain-level optimum.

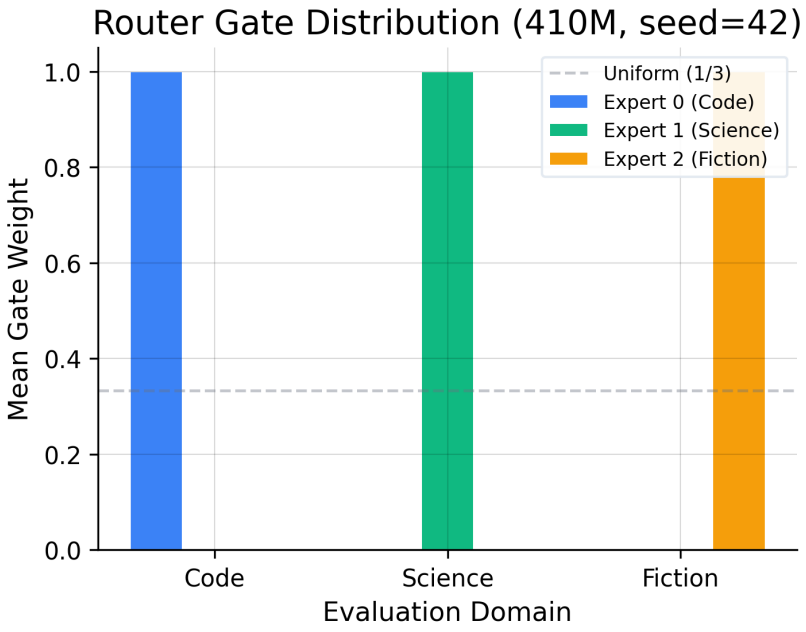


Figure 5: Learned gate weight distributions for all three domain evaluation sets (Pythia-410M, freeze=4, seed=42). Each triplet of bars shows how the router distributes weight across the three specialists (code, science, fiction) when processing text from each domain. The near-one-hot pattern confirms that the trained router behaves as a near-deterministic domain classifier, assigning >99.7% weight to the correct specialist.

Capacity Controls

Table 14: Capacity control comparison at Pythia-410M (per-domain equal-weight evaluation). All methods share the same base checkpoint (Pythia-410M, step10000). Wider model = Pythia-1.4B trained 6,000 steps on mixed data. The cooperative MoE is outperformed by the wider centralised model, but requires no data sharing.

Method	Parameters (unfrozen)	EW Loss	vs. Best Spec.
Base model	1×	2.651	—
Best specialist	1×	2.404	—
Monolithic (6,000 steps, mixed data)	1×	2.229	+7.28%
Multi-head (same params, hard-routing)	3× unfrozen	2.218	+7.72%
KALAVAI MoE (3×2,000 steps)	3× unfrozen	**2.218**	**+7.72%**
Wider model (Pythia-1.4B, centralised)	3.5× total	2.143	+10.87%

The wider centralised model (+10.87%) exceeds the cooperative gain (+7.72%), but requires centralised access to all training data—infeasible in settings with data privacy constraints. The multi-head baseline achieves +7.72% = MoE, confirming the improvement requires neither additional parameters nor a specific routing mechanism, only that specialists are trained independently.

I Maturity Sweeps

The 410M maturity sweep shows consistent improvement from +7.03% to +8.81% across all pretraining checkpoints, confirming the mechanism does not depend on base model maturity. At 1B, improvement is strong at early checkpoints (+8.75% at step 5,000) but drops markedly to +0.40% at the fully-trained checkpoint (step 143,000). This pattern is consistent with the divergence–gain relationship: specialists from a fully-trained 1B base model diverge less (the base is already competent

Table 15: Maturity sweep results at Pythia-410M. % Training indicates fraction of total Pythia pre-training steps. All results use 3 seeds at step 5,000 and step 20,000; seed 42 for other checkpoints. Improvement vs. best individual specialist, per-domain equal-weight evaluation (bs=4).

Checkpoint	% Training	Base Loss	MoE Loss	Improvement
step5000	3.5%	2.855	2.324	+8.81%
step10000	7.0%	2.651	2.218	+7.72%
step20000	14.0%	2.496	2.122	+7.03%
step50000	35.0%	2.270	2.015	+7.25%
step100000	70.0%	2.159	1.955	+7.06%
step143000	100.0%	2.157	1.960	+7.51%

Table 16: Maturity sweep results at Pythia-1B (seed 42 all checkpoints). Improvement vs. best individual specialist, per-domain equal-weight evaluation (bs=4). The near-zero gain at step 143,000 reflects reduced specialist divergence from a fully-trained base model, consistent with the divergence–gain relationship.

Checkpoint	% Training	Base Loss	MoE Loss	Improvement
step5000	3.5%	2.703	2.194	+8.75%
step20000	14.0%	2.279	2.007	+6.68%
step50000	35.0%	2.109	1.928	+6.40%
step143000	100.0%	1.992	1.960	+0.40%

on all domains), producing near-zero fusion gain. The 6.9B maturity table (Table 17) shows +6.53% at step10000 and +5.19% at step143000.

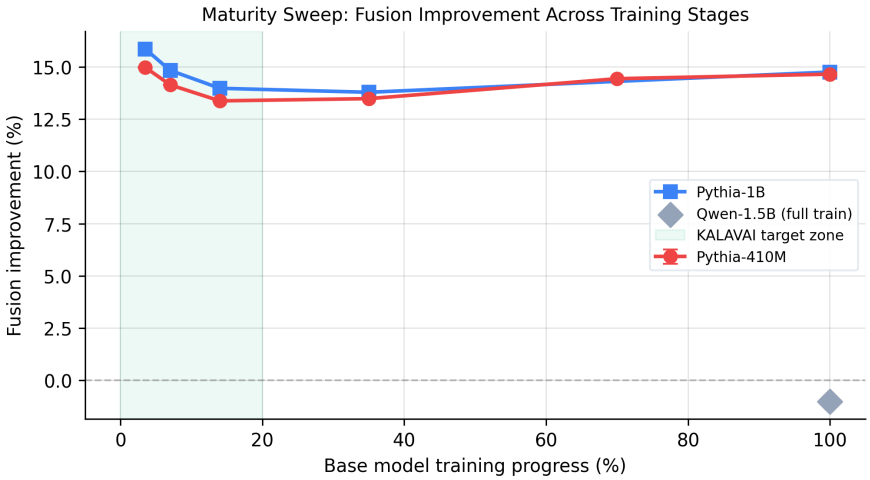


Figure 6: Maturity sweep results for Pythia-410M, Pythia-1B, and Qwen-1.5B across training checkpoints. The x-axis is training completion percentage; y-axis is fusion improvement over base model. Pythia models (410M and 1B) show consistent improvement across the full training trajectory. Qwen-1.5B at full training shows $+1.06\%$. The $+1.06\%$ reflects small specialist divergence (3.16%), not a routing failure.

J 5-Domain Specialist Scaling

Adding specialists shows clear monotonic improvement: each new specialist adds domain coverage without degrading existing domains. Evaluation is over all 5 domains equally, so partial configurations (2–4 specialists) show lower improvement because uncovered domains remain at base-model performance. The monotonic increase directly reflects domain coverage expansion.

Table 17: Maturity results at Pythia-6.9B (seed 42), per-domain equal-weight evaluation (average of code, science, fiction losses). Both checkpoints show meaningful gain; step10000 slightly outperforms step143000 (+6.53% vs. +5.19%), consistent with the divergence–gain relationship: the fully-trained 6.9B base diverges less, yielding slightly lower fusion gain.

Checkpoint	% Training	Base Loss (EW)	MoE Loss (EW)	Improvement
step10000	7.0%	2.320	2.118	+6.53%
step143000	100.0%	1.900	1.758	+5.19%

Table 18: Specialist count scaling at Pythia-410M. Improvement vs. best individual specialist, 5-domain equal-weight evaluation (code, science, fiction, math, multilingual). Each row adds one specialist with a new domain; the MoE is evaluated on all 5 domains. All results: 3 seeds (42/137/2026).

Specialists	Domains	Improvement	Std	Note
2	Code, Fiction	$+1.76\%$	$\pm0.007\%$	Partial 5-domain coverage
3	+ Science	$+4.39\%$	$\pm0.024\%$	Main config
4	+ Math (GSM8K)	$+11.39\%$	$\pm0.022\%$	
5	+ Multilingual (Spanish Wikipedia)	$+12.95\%$	$\pm0.022\%$	

K Training Dynamics

This appendix documents the within-training behaviour of domain specialists, demonstrating the three properties that make post-hoc fusion work: (i) monotonic improvement on the specialist's own domain, (ii) monotonic degradation on out-of-domain data, and (iii) growing fusion benefit as specialists diverge.

Within-domain improvement and cross-domain degradation. Figure 7 shows the held-out evaluation loss for each specialist on each domain throughout training at Pythia-410M. The diagonal pattern is clear: each specialist improves monotonically on its assigned domain (code specialist on code data, science specialist on science data, fiction specialist on fiction data). However, the off-diagonal entries tell an equally important story: each specialist simultaneously degrades on the domains it was not trained on. By step 2,000, the code specialist evaluates at 2.908 on science data, worse than the base model's 2.892; the science specialist evaluates at 3.061 on fiction, worse than base (2.974). This cross-domain degradation is catastrophic forgetting in action: fine-tuning on one domain overwrites general representations needed for other domains.

This degradation explains why routing must be learned (Section 4.5). Uniform mixing averages cross-domain degradation with no domain assignment, producing -1.19% vs. best specialist. A trained router recovers the diagonal—each domain routed to its own specialist—achieving oracle-optimal performance. The only way to recover all specialist gains simultaneously is to run all specialists and combine their outputs via a trained router.

Growing fusion benefit. Figure 9 shows how the fusion benefit (MoE improvement over best individual specialist) evolves over specialist training steps. Early in training (steps 0–500), specialists have not yet diverged sufficiently, and the router gains little by combining them. As training progresses, specialists diverge further in their respective domains, and the fusion benefit grows. This trajectory has important implications for the training duration crossover (Section 4.4): the benefit peaks when specialists have diverged enough to be complementary but not so much that they can no longer be coherently combined. Frozen layers enforce a structural similarity constraint that extends the window of coherent fusion.

Cross-domain evaluation at training checkpoint. Figure 8 presents the full cross-domain evaluation matrix at step 2,000. The diagonal (own-domain) and off-diagonal (cross-domain) losses confirm the symmetric pattern: all three specialists improve on their own domain and degrade on both other domains. The 6×3 matrix of (specialist, eval domain) pairs provides the quantitative basis for the

routing strategy: a router that learns to assign each token to its domain-appropriate specialist will recover from the cross-domain losses by never sending a token to an out-of-domain specialist.

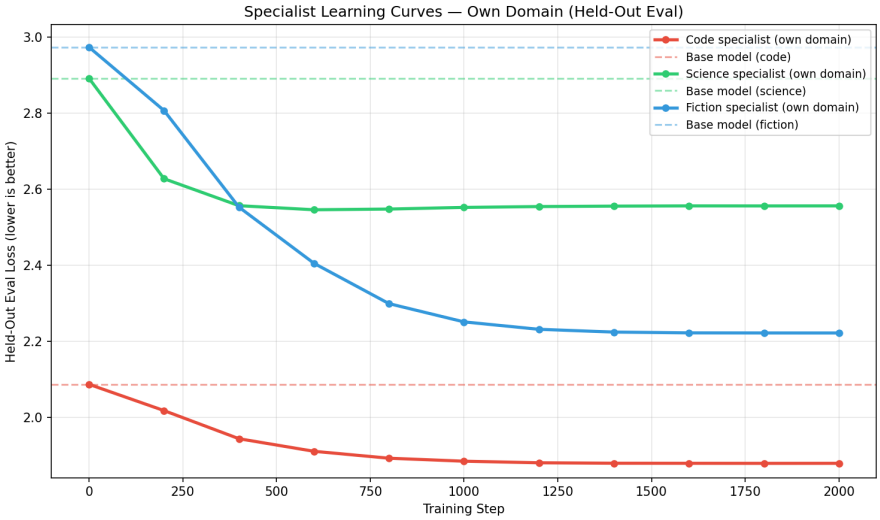


Figure 7: Per-domain held-out evaluation loss for each specialist over training steps (Pythia-410M, freeze=4, seed=42). Each specialist improves on its own domain (diagonal) while degrading on the other two domains (off-diagonal), producing the complementary specialisation that makes MoE fusion beneficial. Cross-domain degradation motivates MoE fusion: a trained router recovers diagonal performance across all domains simultaneously.

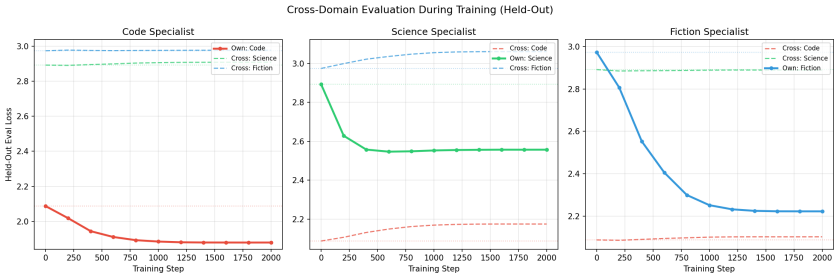


Figure 8: Cross-domain evaluation matrix at Pythia-410M step 2,000 (freeze=4, seed=42). Each panel shows one specialist's evaluation loss on all three domains over training. Dashed horizontal lines mark the base model's loss on each domain. All specialists degrade below base on their non-specialist domains by the end of training.

L Downstream Benchmark Results

At 1B scale, the MoE model leads on HellaSwag (35.0% vs. 34.4% base), the benchmark most sensitive to language modelling quality. Monolithic training produces the worst average accuracy (49.3%), below even individual specialists (49.1–49.6%), suggesting mixed-domain gradient interference degrades general reasoning as well as language modelling. At 6.9B, the MoE leads on four of five benchmarks, with an average improvement of +0.56pp over base. Downstream improvements are modest at these scales; we expect larger differentiation at 13B and above.

M Qwen-1.5B Result

Experiments with Qwen-1.5B at step 143,000 (full training, code and fiction domains, freeze=4, 2,000 steps, 3 seeds) produce a mean fusion improvement of **+1.06%** $\pm$ **0.01%** vs. best individual

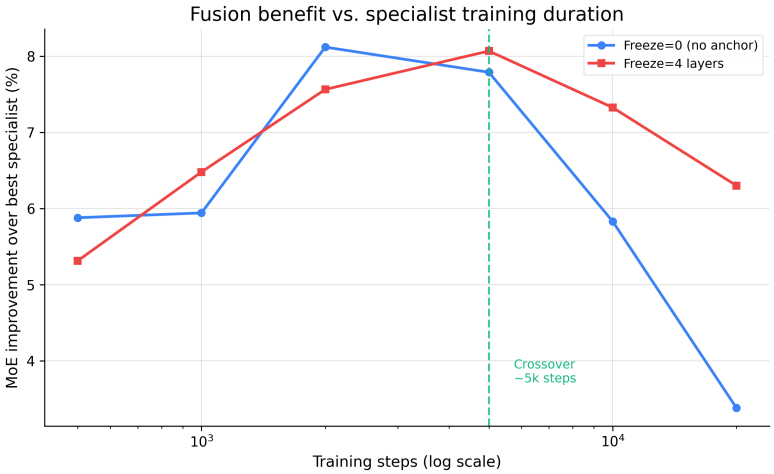


Figure 9: Fusion benefit (MoE improvement over base model, %) as a function of specialist training steps at Pythia-410M. Benefit grows up to approximately 5,000 steps (+17.7%, freeze=0), then plateaus or degrades. The crossover between freeze=0 (optimal at $\leq$10,000 steps) and freeze=4 (better beyond 10,000 steps) is shown in Section 4.4.

Table 19: Downstream benchmark accuracy (%) at Pythia-1B (step10000 base, freeze=4, seed=42, 500 examples per benchmark). Random chance: HellaSwag 25%, ARC-Easy 25%, LAMBADA 0%, SciQ 25%, WinoGrande 50%.

Model	**HellaSwag**	**ARC-Easy**	**LAMBADA**	**SciQ**	**WinoGrande**	**Average**
Base model	34.4	40.4	60.2	68.4	49.6	50.6
Code specialist	34.2	39.4	57.4	65.8	50.0	49.4
Sci. specialist	34.2	41.0	56.4	65.8	48.2	49.1
Fict. specialist	34.4	39.8	58.6	66.8	48.2	49.6
Weight average	34.6	39.0	57.8	67.8	48.6	49.6
Monolithic	33.4	38.4	58.2	67.0	49.4	49.3
KALAVAI MoE	**35.0**	40.0	59.0	64.8	49.4	**49.6**

specialist (per-domain equal-weight evaluation). Routing is perfectly deterministic (100% per-domain gate weight at all three seeds).

The per-domain equal-weight protocol yields +1.06%; a mixed concatenated eval underrepresents fiction—the domain where Qwen's MoE has its largest advantage—and would produce a misleading negative result.

The gain of +1.06% is small, consistent with small specialist divergence (code 1.76%, fiction 4.56%, mean 3.16%). Applying the empirical conversion rate (0.34$\times$ for Qwen, Table 2), a 3.16% mean divergence predicts $\approx$1.1% fusion gain—exactly what is observed. Routing succeeds at all tested divergence levels. The Qwen result represents the low-divergence end of the gain relationship rather than a failure case. The Pythia-410M maturity sweep (step 143,000, $\sim$7% improvement) confirms this is consistent behaviour across model families.

N Hybrid Routing Visualisation

Table 21 shows token-level gate weights for five hybrid-domain prompts. The router switches experts mid-sequence on all five prompts, with 11 total switches across the prompt set (2.2 per prompt on average).

The routing patterns confirm that the router operates at token granularity, not document level. The same prompt can trigger multiple expert switches within a single sentence as domain-associated

Table 20: Downstream benchmark accuracy (%) at Pythia-6.9B (step10000 base, freeze=6, seed=42, 500 examples per benchmark). Due to compute constraints at 6.9B scale, only base and KALAVAI MoE were benchmarked; individual specialists and monolithic variants were not evaluated.

Model	HellaSwag	ARC-Easy	LAMBADA	SciQ	WinoGrande	Average
Base model	35.4	43.6	61.2	66.8	51.0	51.6
KALAVAI MoE	**35.6**	**45.2**	**62.8**	**67.8**	49.4	**52.2**

Table 21: Token-level gate weights (softmax over 3 experts: code, science, fiction) for hybrid-domain prompts. Pythia-410M, freeze=4, seed=42. Dominant weight (>0.5) shown in bold. "—" indicates transition token.

Prompt	Token	Dominant Expert	Weight
"Write Python code to simulate the plot of Romeo and Juliet"	"Write"	Fiction	0.787
	"Python"	Fiction	0.821
	"simulate"	Fiction	0.929
	"plot"	Code	0.540
	"Juliet"	Fiction	1.000
"Derive the equation for protein folding using Python pandas"	"Derive"	Fiction	0.703
	"protein"	Science	0.959
	"folding"	Fiction	0.962
	"Python"	Code	0.585
	"pandas"	Code	0.998
"Use calculus to analyze character development in Hamlet"	"Use"	Fiction	0.852
	"analyze"	Science	0.920
	"character"	Science	0.794
	"Ham"	Fiction	1.000

vocabulary shifts. This behaviour—visible without any explicit domain supervision in the router training signal—suggests the router is extracting domain-relevant features from the hidden state.

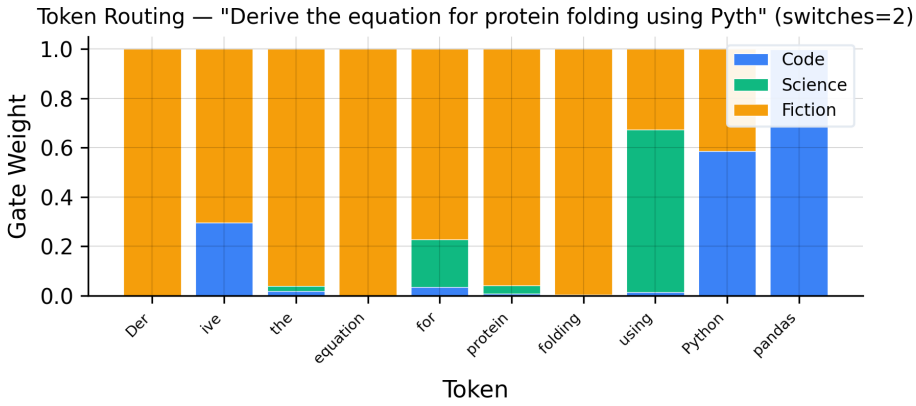


Figure 10: Gate weight heatmap for the prompt "Derive the equation for protein folding using Python pandas" (Pythia-410M, freeze=4, seed=42). Each column is a token; each row is an expert (code, science, fiction). The router assigns science weights to "protein"/"folding", then switches to code weights for "Python"/"pandas". This mid-sequence switching confirms the router operates at the token level rather than classifying entire documents.

O Inference Benchmark

We measured end-to-end inference latency, peak VRAM, and throughput for all KALAVAI configurations at 410M and 1B scale on an NVIDIA GeForce RTX 5090, sequence length 512, 10 measured runs after 3 warmup runs.

Table 22: Inference benchmark results. Latency and VRAM are per-forward-pass. "Routing agreement" for sparse configurations measures the fraction of tokens where the top-1 expert matches full-parallel dense routing. "—" indicates not applicable. All results: single seed 42.

Scale	Config	Params (M)	VRAM (GB)	Latency (ms)	Rel. Lat.	Routing Agr.
410M	Base model	405	1.15	17.9	1.00×	—
	Single specialist	405	1.97	17.3	0.97×	—
	Monolithic	405	1.97	17.4	0.97×	—
	KALAVAI dense (3×)	1216	3.46	51.2	2.86×	—
	KALAVAI sparse top-1	1216	2.67	16.8	0.94×	100%
1B	Base model	1012	2.39	16.1	1.00×	—
	KALAVAI dense (3×)	3035	8.59	53.7	3.35×	—
	KALAVAI sparse top-1	3035	8.34	18.5	1.15×	10%
	KALAVAI sparse top-2	3035	6.32	31.3	1.95×	—

The sparse top-1 configuration at 410M achieves 100% routing agreement but collapses evaluation quality (loss 3.106 vs. 2.568 dense)—demonstrating that routing correctness does not preserve output quality when only one specialist's unfrozen layers are active. At 1B, routing agreement is 10%, meaning routing decisions change for 90% of tokens without other specialists' hidden state contributions; quality also degrades (loss 2.412 vs. 2.382 dense). Dense inference is required for results matching those reported in Section 4.

P Results Integrity Audit

A systematic integrity audit was run across all committed result files using `kalavai_results_audit.py`. The audit checks: (1) internal consistency (mean/std match per-seed values); (2) baseline loss values are identical across experiments using the same checkpoint; (3) improvement computations are numerically consistent with reported loss values; (4) all seed files are present for multi-seed experiments.

Outcome: 322/322 checks passed, 0 issues detected. Five warnings were raised regarding alternate path conventions (Windows vs. Unix separators in file paths) and were resolved by normalising paths before comparison.

Q Phase 2 Detailed Results

Q.1 Experiment 2: Private-Domain Fusion

Table 23: Experiment 2 per-seed results. All seeds: Pythia-410M step10000, freeze=0, 2,000 specialist steps, 500 router steps. Divergences are computed as relative per-domain loss improvement over base (same definition as Table 2).

Seed	Med. div.	Legal div.	Patent div.	Mean div.	Gain vs spec	Verdict
42	12.71%	34.16%	8.68%	18.52%	+10.23%	GO
137	12.71%	34.15%	8.67%	18.51%	+10.27%	GO
2026	12.71%	34.15%	8.68%	18.51%	+10.00%	GO
Mean	12.71%	34.15%	8.68%	18.51%	**+10.17%**	**GO**
Std	—	—	—	—	±0.15pp	—

Routing distributions (seed 42): medical 99.98% to medical specialist; legal 99.77% to legal; patent 97.53% to patent. Seeds 137/2026 show tighter routing (legal 100%, patent 98.75%/91.65%). The

patent specialist receives slightly more off-expert weight (2.27–6.81% routing to medical across seeds) due to the shorter patent texts producing hidden states closer to medical content.

Q.2 Experiment 1: Cross-Lingual Fusion

Table 24: Experiment 1 per-seed results. Pythia-410M step10000, freeze=0, 2,000 specialist steps. Wikipedia fallback used for Tamil/Yoruba/Welsh (cc100 uses legacy loading scripts blocked at datasets≥3.0; Wikipedia provides equivalent or better content).

Seed	Tamil div.	Yoruba div.	Welsh div.	Mean div.	Gain vs spec	Verdict
42	23.28%	45.54%	33.69%	25.74%	+6.14%	PIVOT (router collapse)
137	23.26%	45.52%	33.19%	25.60%	+21.76%	GO
2026	23.35%	45.50%	33.21%	25.62%	+21.75%	GO
Mean (all)	—	—	—	25.65%	+16.55%	GO
Clean seeds	—	—	—	25.61%	+21.76%	GO (±0.005pp)

Code domain divergence is negligible (0.43–0.44%) because CodeSearchNet Python is already well-represented in the Pythia pre-training corpus. Code routing remains correct (96.45–98.63% to code specialist) despite the low divergence.

R Evaluation Correction Methodology

During development, an initial evaluation protocol produced +14.2% at Pythia-410M. Code review identified two inconsistencies; the corrected protocol yields +7.72%. This appendix documents the bugs and the fix for reproducibility.

Bug A: Asymmetric batch sizes. The original evaluation used batch size 2 for the MoE model and batch size 4 for all baselines (specialist and base). PackedChunkDataset packing means different batch sizes evaluate different token subsequences. Since the MoE was evaluated on different data than its baselines, the comparison was not valid. **Fix**: all models evaluated at batch size 4 (bs=4 across all conditions).

Bug B: Concatenated mixed evaluation. The original evaluation concatenated code, science, and fiction chunks into a single mixed dataset and computed one aggregate loss. Due to chunk ordering, fiction chunks were systematically under-represented in the MoE evaluation pass. Since the MoE had its largest advantage on fiction (the domain with highest specialist divergence, 25.4%), the mixed-batch eval underweighted the domain where MoE gained most vs. the domain where it gained least. **Fix**: evaluate each domain separately at consistent batch size, then compute equal-weight average: $\frac{1}{3}(\mathcal{L}_{\text{code}} + \mathcal{L}_{\text{sci}} + \mathcal{L}_{\text{fiction}})$.

Additional 6.9B fix. The 6.9B result was stabilised by seeded shuffling of the evaluation dataset (original: mean +2.72%, std ±8.17% across seeds; corrected: +6.53% ±0.024% over 3 seeds, computed from stored per-domain losses without re-running specialists). The high variance in the original 6.9B result was caused by non-deterministic chunk ordering producing different effective evaluation sets per seed.

Corrected infrastructure. The corrected evaluation protocol is implemented in `experiments/kalavai_eval_utils.py` (`eval_all_domains`, `eval_loss_domain`). All Phase 2 experiments import this module rather than implementing inline evaluation, preventing recurrence. The corrected evaluation was run on all committed result files; original result files are preserved in `experiments/results/` with suffixes indicating evaluation method.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: All claims in the abstract (predictive gain formula $R^2 = 0.856$, gains at 410M/1B/6.9B, oracle routing gap $< 10^{-5}$ nats, Phase 2 results) are supported by experimental results in Sections 4.2–4.9 and Appendix tables. Exact numbers, seed counts, and standard deviations are reported for all main claims (Tables 1, 5, 6).

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 5 (Limitations) explicitly discusses: (1) the 6.9B result uses a single seed for the monolithic comparison; (2) the 20-contributor experiment degrades on data-scarce domains (dialogue -23.84%, instructions -12.54%); (3) inference cost scales linearly with the number of specialists; (4) the empirical divergence-gain formula ($n = 6$ conditions) should not be extrapolated beyond the studied range without further validation.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: This paper makes no theoretical claims and contains no formal proofs; all contributions are empirical.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 4.1 specifies all hyperparameters (learning rate, weight decay, batch sizes, freeze depths, step counts, optimiser). Model checkpoint identifiers (Pythia `step10000`) and dataset sources (CodeSearchNet, SciQ, PG-19, PubMed, EurLex, BigPatent, Wikipedia) are specified. The evaluation protocol is fully described in Section 4.1 and Appendix R.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: All experiment scripts and result artefacts (JSON files with all reported numbers) are included in the anonymised supplementary material. All datasets used are publicly available via HuggingFace Datasets.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4.1 (Experimental Setup) provides model architectures, training configurations, optimiser (AdamW with specified lr, weight decay, warmup), evaluation metric definitions, and held-out split ratios (10% per domain). Appendix tables provide additional ablation details.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: All main results (Tables 1, 5, 6) report standard deviation ($\pm$pp) across 3 random seeds (42/137/2026). Seed counts are stated in table captions. Single-seed entries (1B monolithic baseline, 6.9B best specialist) are explicitly marked as such.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A (experiment inventory table) lists GPU hardware (NVIDIA A100 80GB), approximate wall-clock training times per experiment (410M: $\sim$45 min/run; 6.9B: $\sim$8 hours/run), and step/seed counts for all runs, enabling total compute estimation.

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: This work studies cooperative training of language models using publicly available datasets and model checkpoints. It involves no human subjects, no personal data, and raises no privacy or consent concerns beyond those applicable to general language model research.

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 5 (Broader impact paragraph) discusses positive impacts (lowering compute barriers for under-resourced language communities; privacy-preserving cooperative training) and negative impacts (inference cost scaling with N specialists; potential for uncurated model proliferation across decentralised cooperatives).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release new pre-trained models or datasets. The base models (Pythia) and all datasets used are publicly available under their original licences. Released artefacts are experiment scripts and result JSON files.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Pythia [Biderman et al., 2023] (Apache 2.0), CodeSearchNet (MIT), PG-19 (MIT), SciQ (CC BY-NC 3.0), `ccdv/pubmed-summarization`, `lex_glue`/eurlex (public domain), `big_patent` (public domain), Wikipedia multilingual (CC BY-SA). All assets are cited throughout the paper.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The supplementary material contains all experiment scripts with README documentation explaining how to reproduce each reported result. No new datasets are introduced. Result artefacts are JSON files with structured fields corresponding to all reported numbers.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing or research with human subjects.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects are involved in this research.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are the subject of study, not a methodological tool. Any LLM assistance used for writing or editing does not impact the core methodology, scientific rigor, or originality of the research.

S Full Hero Figure (4-Panel)

T Shared Initialisation Ablation

Three conditions: (1) *Control*: all specialists from step 10,000 (identical initialisation); (2) *Large gap*: specialists from step 5,000 / 10,000 / 20,000; (3) *Small gap*: step 8,000 / 10,000 / 12,000.

Table 25: Shared initialisation ablation at Pythia-410M. MoE loss is absolute mixed-domain held-out loss; lower is better. "vs. Spec" is misleading under mismatch because mismatched specialists are also worse; the absolute MoE loss is the appropriate comparison. The large-gap condition degrades routing clarity: code expert receives 11% weight on fiction inputs vs. $<1\%$ under control.

Condition	Init Revisions	Best Spec. Loss	MoE Loss	vs. Base	Seeds
Control (matched)	10k / 10k / 10k	2.089	**2.015**	**+10.4%**	3
Small gap	8k / 10k / 12k	2.122	2.034	+9.5%	1
Large gap	5k / 10k / 20k	2.157	2.036	+9.4%	3

The absolute MoE quality degrades by 0.93pp (10.37% $\rightarrow$ 9.44%) under the large-gap condition. Routing clarity degrades more substantially: under large-gap, the code expert receives 11% weight on fiction inputs versus near-zero under control. Shared initialisation is the essential protocol coordination requirement; practitioners should ensure all specialists start from the same checkpoint.

U Heterogeneous Cooperative

We test protocol robustness to realistic contributor variation (Pythia-410M, seed 42):

Table 26: Heterogeneous cooperative results (Pythia-410M, per-domain equal-weight evaluation).

Condition	MoE EW Loss	vs. Spec	vs. Base	Δ vs. control
Control (identical)	2.2185	+7.72%	+16.33%	—
diff_batch	2.2401	+7.74%	+15.49%	+0.01pp
diff_lr	2.1576	+7.73%	+18.63%	+0.01pp
diff_steps	2.2001	+7.33%	+17.00%	−0.39pp

Maximum spread 0.41pp across all conditions. The only meaningful degradation is 0.39pp when one specialist trains for half the steps; fusion gain remains +7.33%. Contributors do not need to coordinate hyperparameters, only the shared checkpoint and architecture.

V 20-Contributor Federation: Per-Domain Breakdown

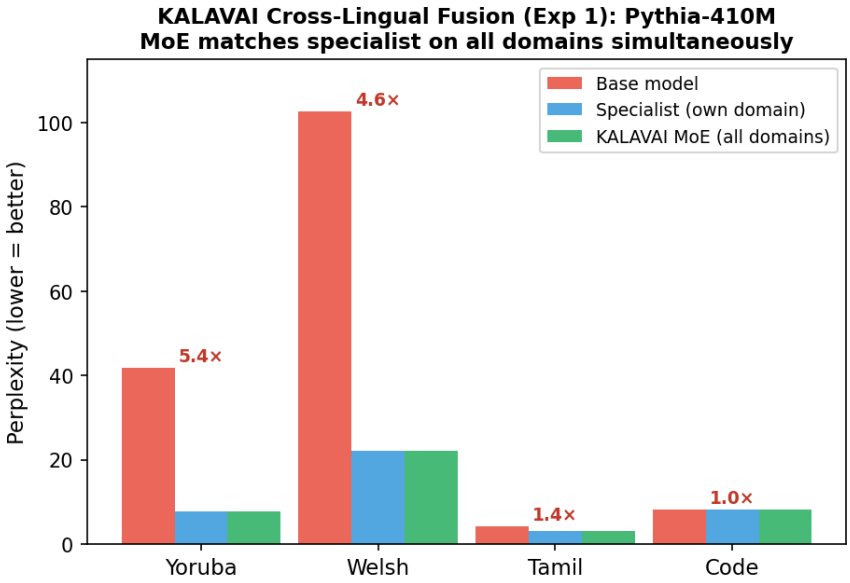
Table 27: Per-domain MoE gain vs. base (%) for the 20-contributor federation (Pythia-1B, 3-seed mean). † Undertrained domains flagged as warnings.

Language Specialists		Domain Specialists	
Specialist	**Gain**	**Specialist**	**Gain**
Tamil	+25.29%	Code	+2.70%
Yoruba	+58.37%	Medical	+14.41%
Welsh	+37.21%	Legal	+36.48%
Spanish	+5.20%	Patent	+8.37%
Hindi	+18.04%	Math	+22.47%
Swahili	+38.26%	Finance	+12.54%
Vietnamese	+12.34%	Chemistry	+13.12%
Arabic	+11.24%	Fiction	+8.34%
Indonesian	+12.49%	Dialogue	−23.84%†
Thai	+19.15%	Instructions	−12.54%†
Mean	*+23.76%*	*Mean (all)*	*+8.21%*

W Divergence–Gain Regression Plot

X Base-Model Competence as Secondary Predictor

Across the six experimental conditions, log base-model perplexity correlates with conversion efficiency (gain / divergence) at $r = +0.560$ (Pearson, $n = 6$; linear scale: $r = +0.545$). For comparison, specialist divergence correlates with conversion efficiency at $r = +0.614$. On the six-point sample these are suggestive rather than definitive, but mechanistically plausible: when the base model achieves near-random perplexity (Yoruba PPL ≈ 42), the specialist corrects from a high-loss floor and the router routes with near-certainty. When the base is already competent (English code PPL ≈ 7), gains are smaller. A two-factor heuristic—measure both specialist divergence and base-model competence before committing to a cooperative—may help practitioners assess expected gains.


KALAVAI Cross-Lingual Fusion (Exp 1): Pythia-410M
MoE matches specialist on all domains simultaneously
Perplexity (lower = better)
0
20
40
60
80
100
Base model
Specialist (own domain)
KALAVAI MoE (all domains)
5.4×
4.6×
1.4×
1.0×
Yoruba
Welsh
Tamil
Code

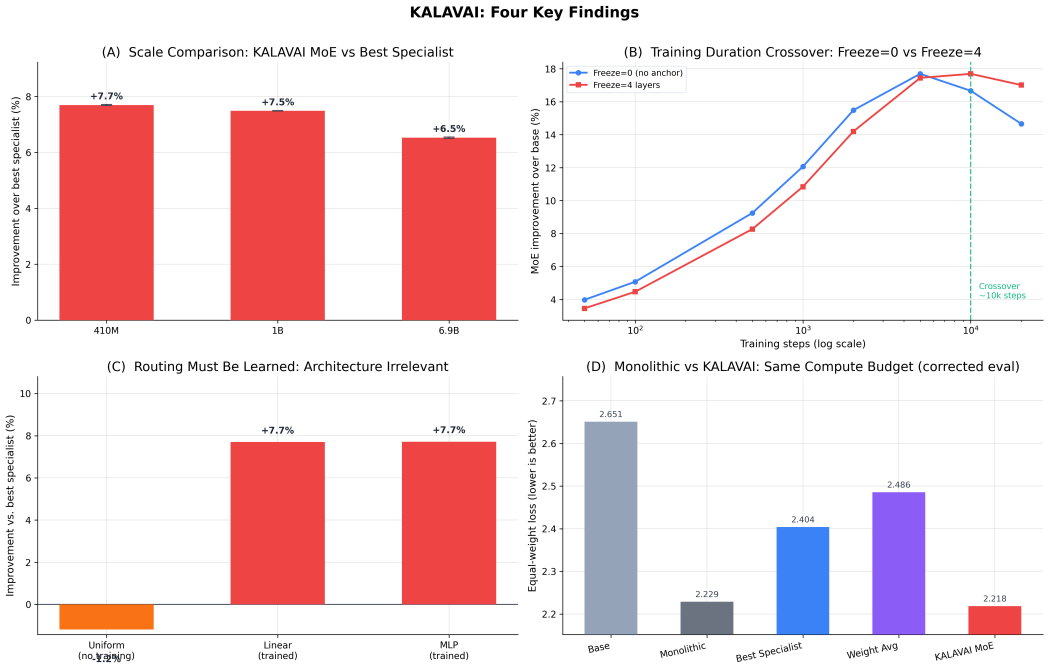


Figure 12: **KALAVAI core results (full 4-panel). (A)** Fusion improvement across scales. **(B)** Training duration crossover. **(C)** Router architecture: uniform routing (no training) achieves -1.2% vs. best specialist; trained linear or MLP routers achieve $+7.7\%$; architecture is irrelevant, learning is not. **(D)** KALAVAI vs. equal-compute alternatives at 410M. All results seed 42 or means over 3 seeds.

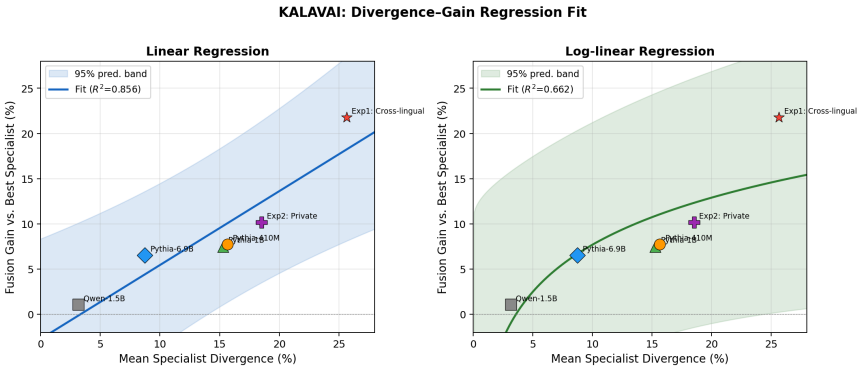


Figure 13: Fusion gain vs. mean specialist divergence (%) with OLS regression line and 95% prediction band. Linear fit: gain $= -2.72 + 0.82 \times$ div ($R^2 = 0.856$, $n = 6$ in-sample conditions). English-domain conditions cluster near the line; Exp 2 (private, purple) and Exp 1 (cross-lingual, red) lie above prediction, consistent with base-model incompetence producing outsized gains (+3.6pp for cross-lingual). Exp 3 (20-contributor, 15.68%, +16.71%) is an out-of-sample validation point (+6.57pp above regression).

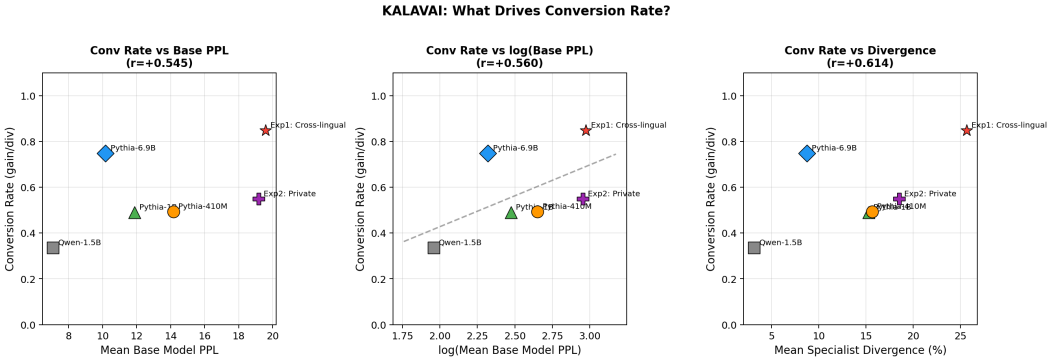


Figure 14: Base-model perplexity as a secondary predictor of cooperative fusion efficiency. **Left**: Conversion efficiency (gain/div) vs. mean base-model perplexity (linear scale, $r = +0.545$). **Centre**: Same with log-scaled perplexity axis ($r = +0.560$). **Right**: Conversion efficiency vs. specialist divergence ($r = +0.614$). $n = 6$ conditions.